

CC3092 Deep Learning y Sistemas Inteligentes

Proyecto 2: Competencia de Agentes en Space Invaders

1. Descripción general

El proyecto consiste en entrenar un agente de Aprendizaje por Refuerzo (Reinforcement Learning) capaz de jugar *Space Invaders* sobre el entorno **ALE/SpaceInvaders-v5** de Gymnasium. El objetivo central no es únicamente obtener el agente con mayor puntaje, sino documentar el proceso de desarrollo e investigación seguido para entrenarlo: qué algoritmos y arquitecturas se probaron, qué funcionó, qué no, y por qué.

El día de la presentación cada estudiante deberá demostrar el desempeño de su agente entrenado. Como parte de la calificación se evaluará el **puntaje (recompensa acumulada)** obtenido por el agente al jugar el entorno, se establecerá un **ranking de la clase** según dicho puntaje, y cada estudiante deberá entregar un **video** que muestre a su agente jugando una partida completa.

Información clave

Modalidad	Individual
Presentación (presencial)	Lunes 21 de octubre de 2026
Entrega del trabajo escrito	Viernes 25 de octubre de 2026
Entorno objetivo	ALE/SpaceInvaders-v5 (Gymnasium + ale-py)
Métrica objetivo	Puntaje promedio.
Modelo a implementar	Agente de Reinforcement Learning (por ejemplo, DQN, Double DQN, Dueling DQN, PPO, A2C, entre otros)
Entregable adicional	Video del agente jugando un episodio completo (.mp4)

Rúbrica

Componente	Puntos	Total
Trabajo escrito	5 pts	5
Competencia (ranking según puntaje promedio en evaluación)	3 pts	3
Total		8

2. Estructura del trabajo escrito

2.1 Definición del problema y análisis del entorno

Presenta un análisis completo del entorno antes de entrenar cualquier agente, análogo al EDA de un problema supervisado.

- Descripción del entorno ALE/SpaceInvaders-v5: objetivo del juego, estructura de recompensa, condiciones de finalización de episodio (terminated/truncated) y número de vidas.
- Espacio de observación utilizado (imagen RGB, escala de grises, RAM, o una combinación) y espacio de acción (Discrete, significado de cada acción).
- Análisis de la señal de recompensa del entorno: ¿es densa o dispersa (sparse)? ¿Requiere algún tipo de normalización o clipping? ¿Hubo necesidad de reward shaping?
- Decisiones de diseño derivadas de este análisis (por ejemplo, número de acciones consideradas, uso de `full_action_space` o solo el subconjunto mínimo, manejo de vidas dentro del episodio) y su justificación.

2.2 Metodología de desarrollo

Describe el proceso seguido para diseñar y entrenar el agente.

- Algoritmo(s) de RL considerados (DQN, Double DQN, Dueling DQN, PPO, A2C) y justificación de la elección final. Puede implementarse desde cero o utilizar una librería, siempre documentando y explicando las decisiones de diseño.
- Arquitectura de la red utilizada para aproximar la política y/o la función de valor (capas convolucionales, capas densas, funciones de activación) y su relación con la dimensión de la observación preprocesada.
- Estrategia de exploración vs. explotación utilizada y sus parámetros.
- Hiperparámetros de entrenamiento: función de pérdida, optimizador, tasa de aprendizaje, tamaño del buffer de repetición de experiencias (*replay buffer*), tamaño de batch, frecuencia de actualización de la red objetivo (*target network*), factor de descuento (γ), entre otros.

2.3 Resultados de iteraciones

Documenta el historial de experimentos realizados durante el entrenamiento del agente, no solo el resultado final. Se recomienda una tabla que resuma cada iteración:

- Identificador de la iteración y cambios realizados respecto a la anterior (algoritmo, arquitectura, hiperparámetros, preprocesamiento, estrategia de exploración).
- Recompensa promedio por episodio obtenida durante entrenamiento y en evaluación (política greedy, sin exploración) para cada iteración.
- Curvas de entrenamiento (recompensa por episodio y/o pérdida vs. pasos de entrenamiento) de las iteraciones más relevantes.
- Evidencia de problemas encontrados (entrenamiento inestable, divergencia de los valores Q, estancamiento en una política subóptima) y cómo se abordaron.

2.4 Discusión de resultados

- Comparación entre las distintas iteraciones: qué cambios de algoritmo, arquitectura o hiperparámetros tuvieron mayor impacto en el puntaje obtenido, y por qué.
- Análisis cualitativo del comportamiento del agente final (con apoyo del video generado): ¿qué estrategias aprendió?, ¿en qué situaciones falla o se queda atascado?
- Discusión de las limitaciones del enfoque utilizado y del cómputo/tiempo disponible para entrenar.
- Reflexión sobre el trade-off entre exploración y explotación observado durante el entrenamiento.

2.5 Conclusiones

- Desempeño final del agente (puntaje promedio de evaluación) y su interpretación en el contexto del juego.
- Principales aprendizajes técnicos y metodológicos del proceso de entrenamiento.
- Posibles mejoras o líneas de trabajo futuro si se contara con más tiempo o recursos de cómputo (por ejemplo, otros algoritmos, más pasos de entrenamiento, ajuste fino de hiperparámetros).

2.6 Enlace al repositorio de GitHub

Incluye el enlace al repositorio público (o con acceso otorgado al curso) que contenga el código completo del proyecto: notebooks o scripts de preprocesamiento del entorno, entrenamiento, evaluación y generación de video, además de un README que explique cómo reproducir los resultados y cómo cargar los pesos del modelo final.

3. Competencia

El día de la presentación cada estudiante deberá tener listo su agente entrenado (pesos guardados) y un script o notebook capaz de cargarlo y evaluarlo de inmediato. La evaluación se realizará de la siguiente forma:

1. Cada estudiante ejecutará su agente durante un número fijo de episodios de evaluación (5 episodios) sobre ALE/SpaceInvaders-v5.
2. Se registrará la recompensa total obtenida en cada episodio y se tomará el mayor de esos 5 episodios para cada estudiante.
3. Con los puntajes de toda la clase se construirá un **ranking** de mayor a menor desempeño. La nota del componente de competencia se asigna en función de la posición relativa de cada estudiante en dicho ranking.
4. Como respaldo de este resultado, cada estudiante debe entregar el video del agente jugando, generado con la misma configuración utilizada en la evaluación en vivo.

Recomendaciones

1. Guardar el modelo entrenado (pesos de la red, arquitectura y configuración de wrappers/preprocesamiento) de forma que puedas cargarlo y generar predicciones rápidamente el día de la presentación.
2. Asegurarse de que el preprocesamiento del entorno usado en evaluación (wrappers, tamaño de frame, número de frames apilados) sea idéntico al usado durante el entrenamiento.
3. Tener un script o notebook listo para cargar el modelo, ejecutar los episodios de evaluación con política greedy, calcular el puntaje promedio y generar el video, sin pasos manuales adicionales.
4. Reutilizar y adaptar las funciones desarrolladas en el Laboratorio #5 (crear_entorno, ejecutar_episodio, generar_video_agente) para la evaluación y grabación del agente entrenado, reemplazando la función de agente aleatorio o de regla simple por la política aprendida.
5. Documentar desde ya cada iteración de entrenamiento: es mucho más fácil escribir la sección de resultados si se lleva un registro continuo (por ejemplo, con TensorBoard, Weights & Biases, o una hoja de cálculo) en lugar de reconstruirlo al final.
6. Iniciar el entrenamiento con anticipación: entrenar un agente de RL sobre juegos de Atari puede tomar considerablemente más tiempo de cómputo que un MLP tradicional.